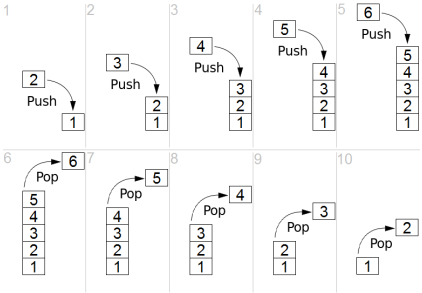
Queues

Reminder: Stacks = LIFOs (Last-In-First-Out)

Stack is an **abstract data type** defined by its three operations:

- `push(x)` puts value `x` on the top of the stack
- `pop()` takes out a value from the top of the stack
  If there are no values in the stack, it raises `EmptyStackException`.
- `isEmpty()` says whether the stack is empty

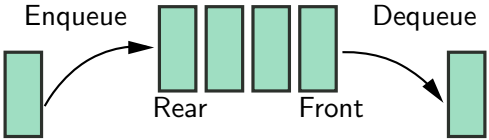


(picture source: wikipedia)

Queues = FIFOs (First-In-First-Out)

Queue is an abstract data type defined by its three operations:

- `enqueue(x)` puts value `x` at the *rear* of the queue
- `dequeue()` takes out a value from the *front* of the queue
  If there are no values in the queue, it raises `EmptyQueueException`.
- `isEmpty()` says whether the queue is empty



Example

Starting from an empty queue, enqueue 4 and 3, dequeue, then enqueue 1 and dequeue two times. What is the last value you get? What would happen in the next dequeue?

Usage

A typical application of queues is a print queue: files are sent to the queue for printing and are printed in the order in which they were sent.

After sending a file, you know that only the jobs currently in the queue will be printed before yours.

Queue ADT

Here is a possible list of operations for a Queue ADT (many variations are possible).

Constructors and Accessors:

- `EmptyQueue`: returns an empty Queue
- `enqueue(value)`: pushes an element to the *rear* of the given queue.
- `dequeue()`: returns the value of the element in *front*, and removes it from the queue.[1]
- `isEmpty()`: reports whether the queue is empty

[1] Triggers error if the queue is empty

Queue as a linked list

In order to have an efficient implementation we need to store the location of the last element in the linked list.

Remember: dequeue from **front**, enqueue to **rear**

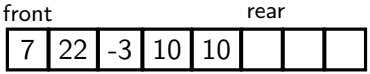
We have two options again:

1. Front at beginning of the linked list, rear at end
2. Rear at beginning of the linked list, front at end

Question: Which one is better and why?

Queue stored in an array (1st attempt)

Whenever we `enqueue` (resp. `dequeue`) we increment `rear` (resp. `front`):



Queue stored in an array (1st attempt)

Whenever we `enqueue` (resp. `dequeue`) we increment `rear` (resp. `front`):

front						rear	
7	22	-3	10	10	4		

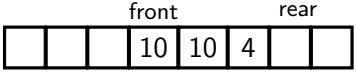
Queue stored in an array (1st attempt)

Whenever we `enqueue` (resp. `dequeue`) we increment `rear` (resp. `front`):

	front					rear	
	22	-3	10	10	4		

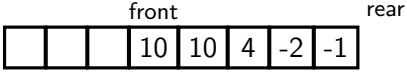
Queue stored in an array (1st attempt)

Whenever we `enqueue` (resp. `dequeue`) we increment `rear` (resp. `front`):



Queue stored in an array (1st attempt)

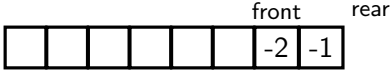
Whenever we `enqueue` (resp. `dequeue`) we increment `rear` (resp. `front`):



We eventually run out of space! To fix this, every time we `dequeue`, move everything to the left. It works but is slow!

Queue stored in an array (1st attempt)

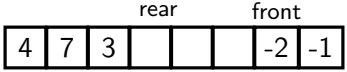
Whenever we `enqueue` (resp. `dequeue`) we increment `rear` (resp. `front`):



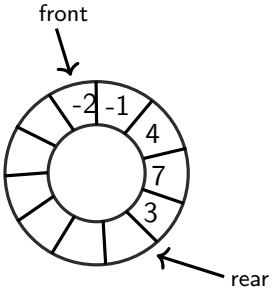
We eventually run out of space! To fix this, every time we `dequeue`, move everything to the left. It works but is slow!

Queue stored in an array (1st attempt)

Whenever we `enqueue` (resp. `dequeue`) we increment `rear` (resp. `front`):



We eventually run out of space! To fix this, every time we `dequeue`, move everything to the left. It works but is slow!

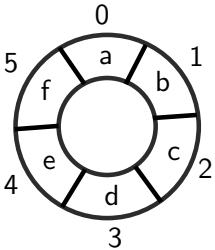


Instead we use a circular storage of a queue. We move `front` and `rear` clockwise.

Works beautifully in theory but how do we implement it?

Representing circles

Positions in the circle, numbered clockwise, correspond to the positions in the array:



$\Longrightarrow$

0	1	2	3	4	5
a	b	c	d	e	f

Example

For a position `pos` in the circle (e.g., `pos = 5`), moving clockwise by one positions is computed as `(pos + 1) mod 6`; in general `(pos + 1) mod circle_length`.

We use `mod` sign to represent the modulo operator, e.g., 11 mod 6 = 5.

Mod and Div in Java

The math operators mod and div are defined so that $x \bmod y \in \{0, \ldots, y-1\}$, $x \operatorname{div} y \in \mathbb{Z}$, and

$$x = (x \operatorname{div} y) \times y + (x \bmod y).$$

Java's `%` and `/` interpret the operations on negative numbers incorrectly (Java, and C, allows negative results for mod). Instead, use `Math.floorDiv` and `Math.floorMod`.

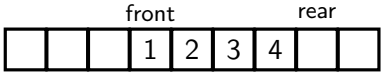
For example:

- `127 % 10 == 7` and $127 \bmod 10 = 7$,
- `-18 % 10 == -8` and `-18 / 10 == -1`, but
- but $-18 \bmod 10 = 2$ and $-18 \operatorname{div} 10 = -2$.

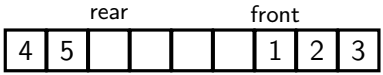
Circular queue: Array implementation

We store values in a `queue` array in between positions marked by `rear` and `front`, i.e.,

- if $\texttt{rear} \geqslant \texttt{front}$ then the queue consists of the entries on positions $\texttt{front}, \texttt{front} + 1, \ldots, \texttt{rear} - 1$:



- if $\texttt{rear} \leqslant \texttt{front}$ then the queue consists of the entries on positions $\texttt{front}, \texttt{front} + 1, \ldots, \texttt{MAXQUEUE} - 1$ and $0, 1, \ldots, \texttt{rear} - 1$:



Digression: Invariants

An invariant is a condition on code. It must always be true during the execution of some section of code, e.g., a loop, or during the execution of a method, or, if it applies to a class, then it can be a condition that must be met by all objects of the class on every entry to and exit from the methods of the class even if it can be temporarily false during the execution of those methods.

Invariants are important because they specify conditions that must be met and maintained in parts of the code. So not only do they communicate information to the reader of the code, but they are tools that can be used both to identify and debug errors in the code (e.g. print an error message if this invariant is broken, stop in the debugger at any point when this invariant becomes false, etc.), and can be used to mathematically prove that the program is correct.

Circular queue: Array implementation

The invariants maintained in this implementation are:

- $0 \leqslant$ `front` $<$ `MAXSIZE`
- $0 \leqslant$ `size` $\leqslant$ `MAXSIZE`

Note that rather than using a *rear* index variable, we will always calculate it when we need it from *front*, *size* and *MAXSIZE* since

```
rear = (front + size) % MAXSIZE
```

The queue is full if `size == MAXSIZE` and empty when `size == 0`

To enqueue, we first check that the queue is not full, put the new value at index position `rear`, and increment `size` by adding 1 to it.

To dequeue, we first check that the queue is not empty, get the value at index position `front`, decrement `size` by 1, and increment `front` by calculating:

```
front = (front + 1) % MAXSIZE
```

```java
class Queue {
  int[] queue;
  int front;
  int size;

  public Queue() {
    // initialize empty queue
    this.queue = new int[MAXSIZE];
    this.front = 0;
    this.size = 0;
  }

  boolean isEmpty () {
    return this.size == 0;
  }

  int dequeue () {
    if (this.size == 0)
      throw new QueueEmptyException;
    int value = this.queue[front];
    this.front = (this.front + 1) % MAXSIZE;
    this.size--;
    return value;
  }
  void enqueue (int value) {
    if (size == MAXSIZE) {
      throw new QueueFullException;
      // or allocate more memory
      // and increase MAXSIZE
    }
    // % is okay here since front + size >= 0
    int rear = (this.front + this.size) % MAXSIZE;
    this.queue[rear] = value;
    this.size++;
  }
}
```

Double Ended Queues: Deques

While the Java library does have a `Stack<...>` class, it is an old design that has been kept for backwards compatibilty purposes and should not normally be used. For both `Stack` and `Queue` classes, you should use the `Deque<...>` class, which implements a double-ended queue data type. This has implementations `ArrayDeque<...>` and `LinkedList<...>` and supports inserting at and removing from both ends.

Actually, the `Deque<...>` class is really a Java Interface, rather than a full Class. This will be covered in your Java programming module but the distinction is not important for the purposes of this module.

Final points

As we will see, stacks and queues are used in many algorithms.

We often just say "make a stack" or "make a queue" and we don't care how they are implemented.

We know that, whether it is as an array or linked list, it can be done efficiently.